AI に必要な数学の基礎

１．関数

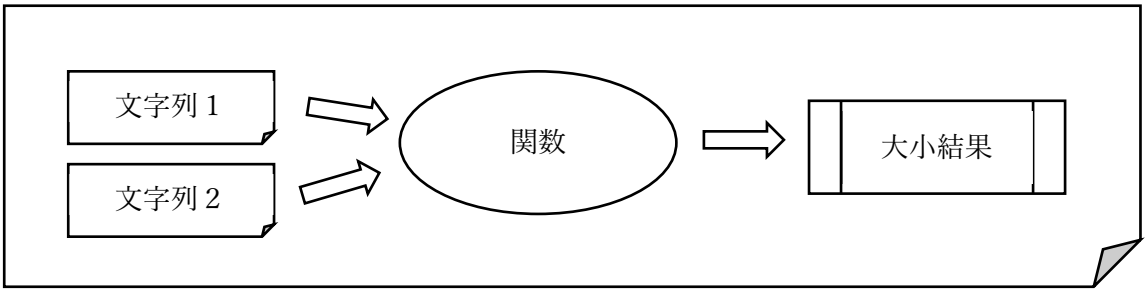
関数…C 言語のプログラムでも関数という言葉が出てきています。簡単に言えば、入力されたデータを処理して出力する働きを持つものです。

C 言語のプログラムを例に挙げると、文字列を比較する関数がありましたね。

入力　２つ　比較する文字列データ

処理　文字列を比較する

出力　文字列の大小の結果（英和辞典で言うと掲載ページの前後）



もともと数学の考えです。数学ではどのように扱っているのかを説明します。

文字式で表現するのでこの関数を f で表現します。（関数を英語で function というのでその頭文字）

入力はこの f に（　）を付けて指定します。例えば、入力が x であれば、$f(x)$ と表します。

処理はどのように表すのかというと、$f(x)$ の処理は…　という感じで

$f(x)$ ＝ とし、続いて処理である数式を記述します。処理が　入力値を 5 倍して 3 を加える　なら

$f(x) = 5x + 3$

と表します。

出力は、$f(x)$ そのものです。

例えば上記の例でいうと　入力 x が　4　の時、

$f(4) + 7$ の出力は、

$f(4) + 7$

$= 5 \times 4 + 3 + 7$　　（$f(x) = 5x + 3$ の x の部分に 4 を代入します。）

$= 20 + 3 + 7 = 30$

となります。

この出力を他の文字式に代入したい場合は、$y = f(x)$ と記述します。プログラムと一緒ですね。

上の例でいうと

$y = f(x)$ は $f(x) = 5x + 3$ なので　$y = 5x + 3$ ということになります。

入力が複数ある場合は、（　）の中に、入力値を　，で区切って記述します。

例えば、入力値　x と y で、処理が、 $3x + 7y - 4$ 、出力値を z に代入したい場合は

$f(x, y) = 3x + 7y - 4$

$z = f(x, y)$

のように記述します。

・入力と出力の関係を表す

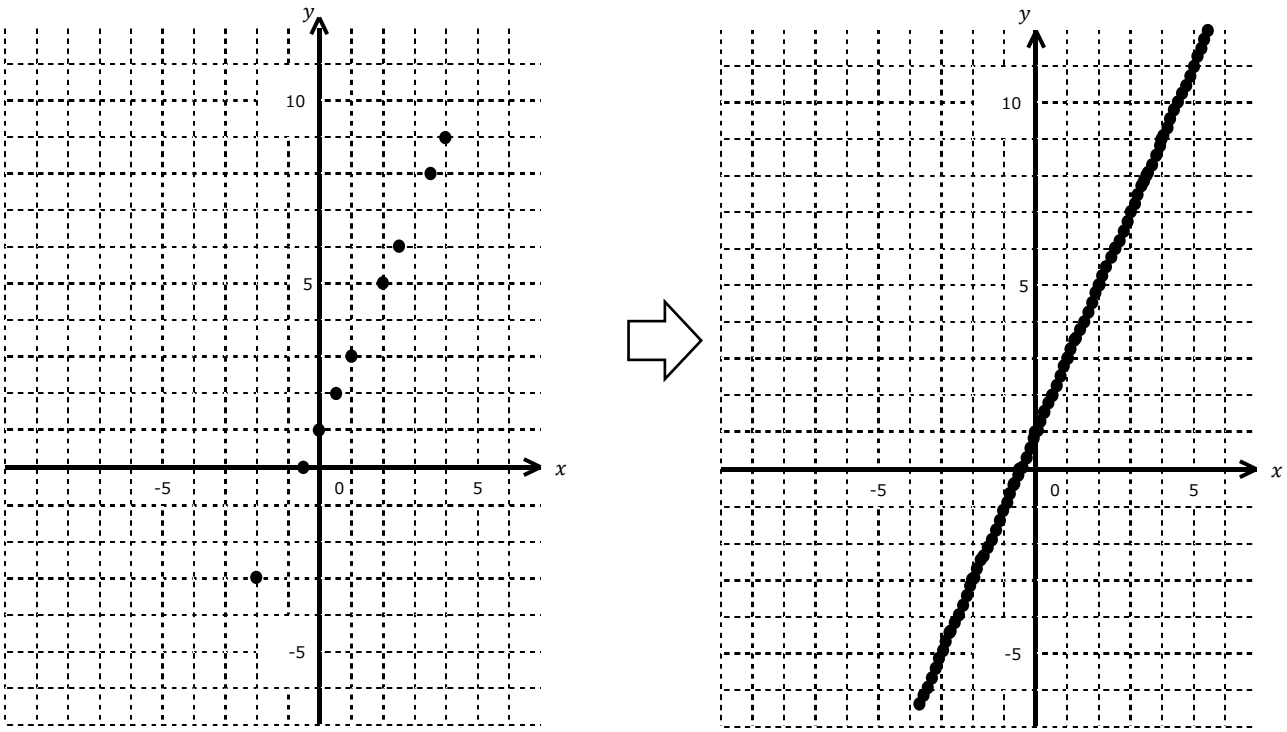
数学の場合、処理が数式なので、入力と出力の関係にある法則が現れてきます。それをビジュアル化したのがグラフとなります。

例えば、$f(x) = 2x + 1$ とし、$y = f(x)$ の場合を考えて見ましょう。

x にいろいろな値を入れて y の値を計算し、表示してみました。

x	-2	-0.5	0	0.5	1	2	2.5	3.5	4
y	-3	0	1	2	3	5	6	8	9

横軸に x 縦軸に y を設定しグラフ上に表してみます。⇒ さらに細かいデータを表していくと…



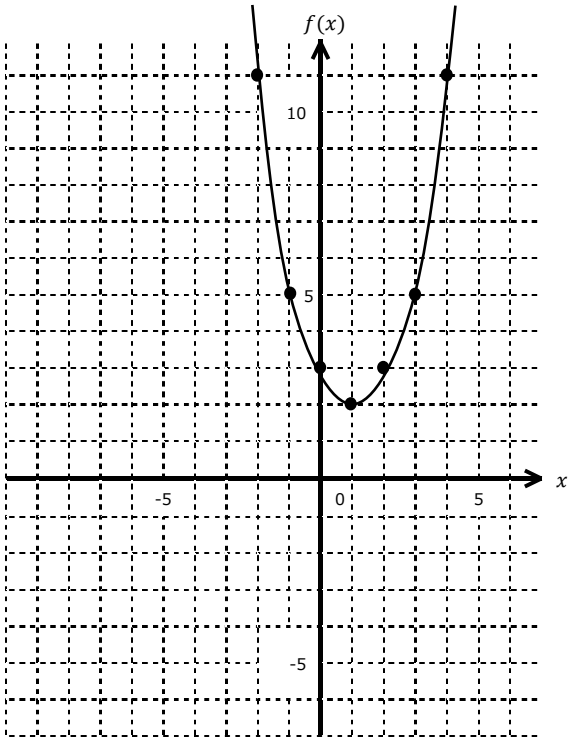
このように直線が現れてきます。

グラフ化されると、入力と出力の関係がよくわかります。

その他もやってみましょう。

$f(x) = x^2 - 2x + 1$

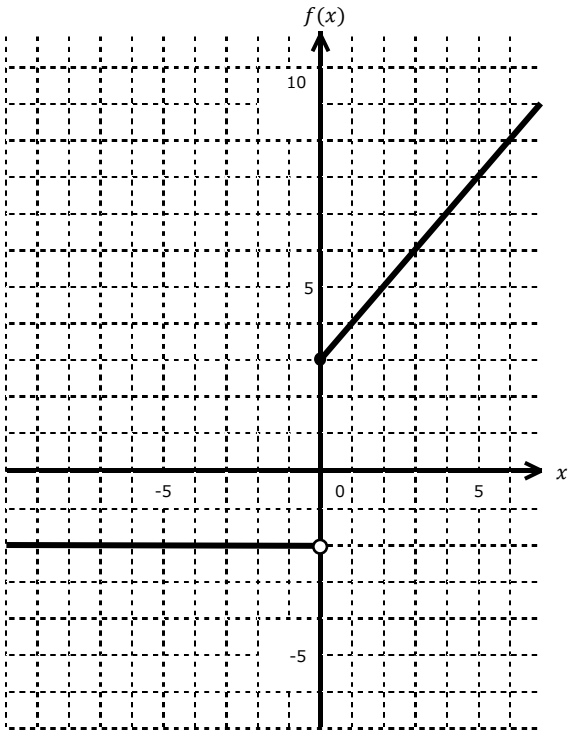
これは放物線と呼ばれているものです。



これはどうでしょう？

$$f(x) = \begin{cases} -2 & (x < 0) \\ x + 3 & (x \geqq 0) \end{cases}$$

x の範囲で処理が変わるというものです。
x が 0 より小さい時 $f(x) = -2$ となります。これは
$f(-1)$ も -2 $f(-100)$ も -2 となります。入力値に影響されずに出力は -2 になるということになります。
グラフで表すと右図のようになります。
$x = 0$ の所を見てください。
○と●があります。
この○は、その点は含まないということを意味し、●はその点を含むということを表しています。
同じ値で複数の出力があったらおかしいですよね。
この場合、$x = 0$ の時は、$x + 3$ の処理がされ、出力は 3 となります。



２．数列・数列の和

ここでクイズです。 □の中には何がはいるでしょうか？

１ ・ ４ ・ ７ ・ □ ・ １３ ・ １６ ・ …

どのように考えましたか？まず、それぞれの数字の関係性を探ったと思います。

「どうも、隣どうしの数字には ３ずつ の差がある」ということが分かってきます。「□は７の隣だから ３を加えると１０になるな。仮に１０として、10に３を加えると １３になり おっ！ まさしくその隣は １３」となり、□が１０であることをつきとめることができました。

このように、あるルールに従った数の列を **数列** といいます。

並べられた数字の１つ１つを **項** といいます。1 番目の項を **初項** 、2 番目の項を **第２項** 、3 番目の項を **第３項** そして n番目の項を **第n項** といいます。

～ n について～

数式などでよくアルファベット文字が使用されます。無限に存在するものや、複雑な計算式などを文字に置きかえて簡単にするためにです。例えば、数列の項ですが第 1 項、第 2 項、第 3 項、第 4 項とこの説明は無限に続きます。この数字の部分をnと置き換えて、「n番目の項は第n項という」 というようにします。この時 n は、1 以上の整数（自然数）を表します。（一般的に数学では自然数を n と表します。）「n番目は…」 の n と「第n項という」の n は 同じ数字を表します。

※ 関数の $f(x)=3x+5$ も $f(x)$ の x と $3x+5$ の x は同じ数字を表します。

では、次に一般項について説明します。

先ほどのクイズでは、第 1 項は １、第 2 項は ４、となりますね。

第n項は、どのように表したらいいでしょうか？

１ ・ ４ ・ ７ ・ １０ ・ １３ ・ １６ ・ …

+3 +3 +3 +3 +3

初項 1

第2項 1 ＋ 3

第3項 1 ＋ 3 ＋ 3

第4項 1 ＋ 3 ＋ 3 ＋ 3

第5項 1 ＋ 3 ＋ 3 ＋ 3 ＋ 3

第6項 1 ＋ 3 ＋ 3 ＋ 3 ＋ 3 ＋ 3

1 番目の項は３が０個
1 番目の項は３が１個
1 番目の項は３が２個
1 番目の項は３が３個
1 番目の項は３が４個
1 番目の項は３が５個

３の個数は、項の番号より１小さい数

なので、

初項 $1+3\times(1-1)$

第2項 $1+3\times(2-1)$

第3項 $1+3\times(3-1)$

第4項 $1+3\times(4-1)$

第5項 $1+3\times(5-1)$

第6項 $1+3\times(6-1)$

と、計算式をまとめます。では、 第n項は…？

$1+3\times(n-1)$ と表すことができます。

数列全体を表したいときは、この n と 集合を表す $\{\quad\}$ を使って

$\{a_n\}$ などと表現します。（a は数列名）

この例の場合、

$\{a_n\} = 3(n-1)+1$

と、表します。

$\{a_n\} = 1+3 \times (n-1)$ では、だめなのか？　だめではないけど、通常は

１．掛け算・割り算の式は、足し算・引き算より先に表現する。

２．数字と文字式の掛け算は、× を省略する

３．数字と文字式では数字を先に表現する。

としています。この暗黙の了解といわれるようなルールを守ることによって誰が見てもわかりやすい式となりますので、皆さんも慣れていきましょう。

この一般項を使って別の表し方で数列を表現することもできます。

隣り合う項の関係で表現する方法です。これを数列の帰納的定義といいます。

たとえば、先ほどの数列を考えてみましょう。一般項は、

$\{a_n\} = 3(n-1)+1$ でしたが、項の値は、1 つ前の項に 3 を加えたものでした。ですから、次の式が成り立ちます。

$a_n = a_{n-1}+3$ (n番目の項は、一つ前、すなわち $n-1$ 番目の項に３を加えたもの)

初項は 1 なので $a_1 = 1$ と表現できます。

a_1が分かれば、 a_2 が分かります。a_2 が分かれば、a_3 が分かります。このように 1 つの具体的な事象から、推論できることを帰納といいます。

このように、初項と各項の関係式で、数列を表現することができます。

この関係式の事を 漸化式 といいます。

2 つの漸化式が、互いの項を参照し、数列を形成するということもできます。連立漸化式と呼ばれます。

例えば、

$a_1 = b_1 = 1$

$$\begin{cases} a_{n+1} = a_n + 2b_n + 2 \\ b_{n+1} = 2a_n + 3b_n + 1 \end{cases}$$

といった場合です。この場合でも帰納的に項を求めることができます。

$$\begin{cases} a_2 = a_1 + 2b_1 + 2 = 1 + 2 \times 1 + 2 = 5 \\ b_2 = 2a_1 + 3b_1 + 1 = 2 \times 1 + 3 \times 1 + 1 = 6 \end{cases}$$

$$\begin{cases} a_3 = a_2 + 2b_2 + 2 = 5 + 2 \times 6 + 2 = 19 \\ b_3 = 2a_2 + 3b_2 + 1 = 2 \times 5 + 3 \times 6 + 1 = 29 \end{cases}$$

この関係は AI のニューラルネットワークにも利用されているので、しっかりと理解をしましょう。

３．∑　記号

∑　は「シグマ」と読むギリシャ文字です。アルファベットの「S」に対応します。「和」（Summary）の頭文字となります。

使い方（表現方法）を説明します。

例えば、1～10 までの整数の合計を表す方法を見てみましょう。

$1+2+3+4+5+6+7+8+9+10$ と表現する方法があります。10 までならこの方法でもいいかもしれませんが、10,000 となると、ちょっと難しくなります。このように順に並んでいる数字（数列）を合計していく計算式を ∑　で表すことができます。

$1+2+3+4+5+6+7+8+9+10$ は

$$\sum_{k=1}^{10} k$$ と 表すことができます。

意味は k を　1 から順に 10 まで変化させてできる数列を順に足していく　ということになります。

この場合、数列は、1,2,3,4,5,6,7,8,9,10 となります。この数列を順に足していくということなので、

$1+2+3+4+5+6+7+8+9+10$ となります。

$$\sum_{k=1}^{7}(2k+3)$$ の場合は

$2\times1+3,\ \ 2\times2+3,\ \ 2\times3+3,\ \ 2\times4+3,\ \ 2\times5+3,\ \ 2\times6+3,\ \ 2\times7+3$ となり、この数列は

5,7,9,11,13,15,17 ということなので、

$5+7+9+11+13+15+17$ となります。

数列の一般項を使って表現してみます。

数列 $\{a_n\}$ の項をすべて合計した式は、

$$\sum_{k=1}^{n} a_k = a_1 + a_2 + a_3 + a_4 + a_5 + \cdots + a_{n-2} + a_{n-1} + a_n$$

となります。

この ∑　記号には線形性と呼ばれる性質があります。

$$\sum_{k=1}^{n}(a_k+b_k) = \sum_{k=1}^{n} a_k + \sum_{k=1}^{n} b_k$$

$$\sum_{k=1}^{n} ca_k = c\sum_{k=1}^{n} a_k \quad (c\text{は定数})$$

この性質を使用し、式を変形し計算することができます。

例えば

$$\sum_{k=1}^{n}(k^2-3k+2) = \sum_{k=1}^{n} k^2 - 3\sum_{k=1}^{n} k + \sum_{k=1}^{n} 2$$

となります。

ちなみに、

$$\sum_{k=1}^{n} 2$$

は、

$k=1$ の時は　2、$k=2$ の時は　2、$k=3$ の時は　2、$k=4$ の時は　2と k の値に関係なく　2　となり、

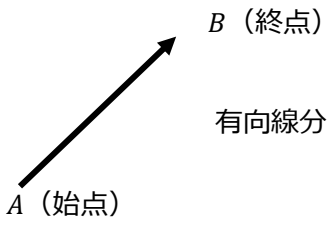
数列は　2,2,2,2,2,2,2,2⋯,2,2,2　（　2　が n 個）となります。

４．ベクトル

大きさと向きを持つ量として定義され、矢印で表現されます。

２点 A, B があるとき、A から B へ向かう線分を考えます。

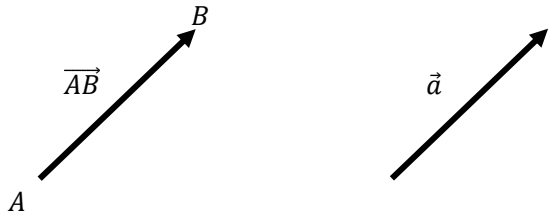
この向きを持つ線分 AB を有向線分といいます。 A を 始点、 B を 終点 と呼びます。



有向線分の３つの属性は　始点Aの位置 ・ 終点Bへの向き ・ AB の長さ（大きさ）　です。

この３つの属性のうち、向きと大きさだけを抽象した量を ベクトル と呼びます。

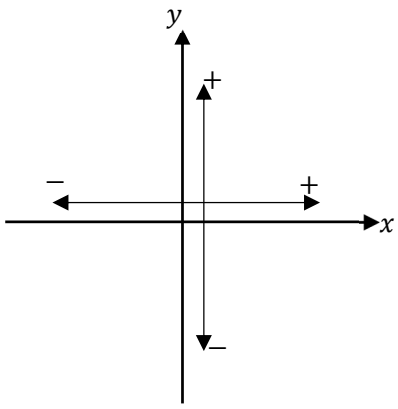
ベクトルは、始点 A と、終点 B を用いて、 $\overrightarrow{AB}$ と表したり、抽象した有向線分として $\vec{a}$ と表したりします。

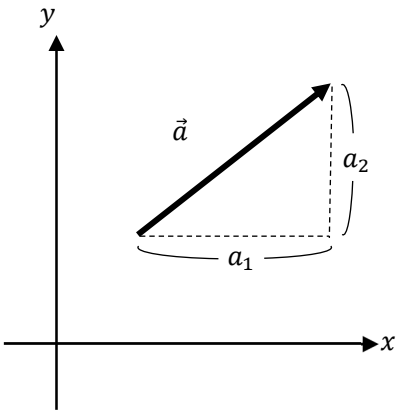


このベクトルを座標系に置くことで、ベクトルは表現できます。

平面上であれば、横方向と縦方向の大きさで表します。横方向は、右が正で左が負、縦方向は上が正で下が負です。

平面座標では、横方向は、x軸、縦方向は y軸となります。

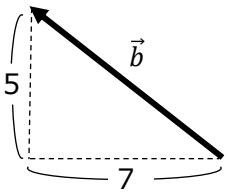




上図右側のベクトル $\vec{a}$ を x軸、 y軸 それぞれの方向の大きさで次のように表現します。

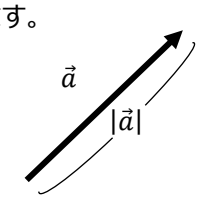
$\vec{a} = (a_1, a_2)$　　これをベクトルの成分表示といいます。

たとえば、下図の $\vec{b}$ は x軸方向が左になるので負となり



$\vec{b} = (-7,5)$ となります。

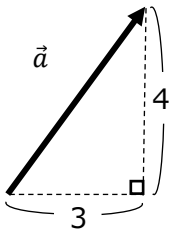
直感的には、ベクトルを表す矢の長さを、そのベクトルの大きさといいます。
ベクトル $\vec{a}$ の大きさを $|\vec{a}|$ と表します。



たとえば、
$\vec{a} = (3,4)$ の大きさは、三平方の定理（ピタゴラスの定理）より

$|\vec{a}|^2 = 3^2 + 4^2$ よって

$|\vec{a}| = \sqrt{3^2 + 4^2} = \sqrt{9+16} = \sqrt{25} = 5$ となります。

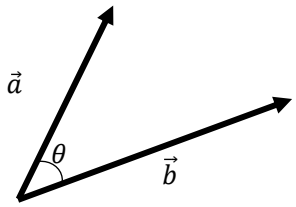


ベクトルは大きさだけでなく、向きもあります。掛け算をする場合、その向きも考慮しなければなりません。
そこで、新たな定義が導入されました。定義なので、このように計算することで、結果の区別をする。ということで受け入れましょう。
たとえば、$\vec{a}$ と $\vec{b}$ の掛け算を考えます。通常の掛け算の場合、向きを無視して、それぞれの大きさを掛け算すれば求まります。

$|\vec{a}||b|$ となります。

大きさが同じだと、向きが違っていても同じ値になります。そこで、2 つのベクトルのなす角度を計算式に組み込みます。角度の関係を組み込む必要があるので、三角関数の cos を使います。
なす角度を θ とし、そこで決められた定義は

$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}|\cos\theta$



最初の ・ は、ベクトルの掛け算を意味し、これを 内積 といいます。
内積にはベクトルの成分で計算する方法もあります。

$\vec{a} = (a_1, a_2)$、$\vec{b} = (b_1, b_2)$ とするとき、

$\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2$

となります。
平面（二次元）の場合、成分は２つですが、数学的には何次元でも可能になります。一般的なベクトルと内積の計算式は、以下のようになります。一般的なので、n 次元で考えます。

$\vec{a} = (a_1, a_2, a_3, \dots, a_n)$、$\vec{b} = (b_1, b_2, b_3, \dots, b_n)$ とするとき、

$\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$ となります。

また、$\cos\theta$ の取りうる範囲は $-1 \leqq \cos\theta \leqq 1$ なので、

$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}|\cos\theta$ より

$-|\vec{a}||\vec{b}| \leqq \vec{a} \cdot \vec{b} \leqq |\vec{a}||\vec{b}|$ （コーシー・シュワルツの不等式）

となります。
ニューラルネットワークの計算では、ベクトル的な見方が役に立ちます。

５．行列

連立漸化式で例として挙げていた式

$$\begin{cases} a_{n+1} = a_n + 2b_n + 2 \\ b_{n+1} = 2a_n + 3b_n + 1 \end{cases}$$

ですが、a_{n+1} を求めるのに、１つ前の項がどの程度影響するか（a_n はそのまま加算、b_n は 2 倍して加算）。
その情報だけを、管理する方法があります。
この例では、

a_n が影響する　b_n が影響する　加算分

$$A = \begin{pmatrix} 1 & 2 & 2 \\ 2 & 3 & 1 \end{pmatrix}$$

これは、関数で言うと、計算（処理）するためのパラメータ（設定値）ということになります。
このように、パラメータなどを、並べ、(　) でまとめたものを　行列　といいます。
横の並びを　行　縦の並びを　列　といいます。この例では、2 行 3 列の行列といいます。
A は、行列の名前になります。アルファベットの大文字で行列を表すのが一般的です。
行列を一般的に表現すると、（行列の名前を A とします。 m 行 n 列）

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

i 行 j 列 に位置する値（成分）を a_{ij} として表しています。
特に成分 a_{ii} （行の位置番号と列の位置番号が同じ）が 1 で、他の成分が 0 の行列を単位行列といいます。
単位行列は通常 E で表されます。（Eはドイツ語で 1 を表す ein の頭文字）

2 行 2 列の単位行列　　3 行 3 列の単位行列

$$E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \qquad E = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

行列の性質

・等しい　行列が等しいということは、対応する各成分が等しいことを意味します。
　　行列 A と、行列 B が等しい場合、$A = B$ と書きます。
・和と差　2 つの行列 A, B の 和 $A + B$、差 $A - B$ は、同じ位置の成分どうしの和、差と定義されます。
・定数倍 行列の定数倍は、各成分を定数倍したものと定義されます。

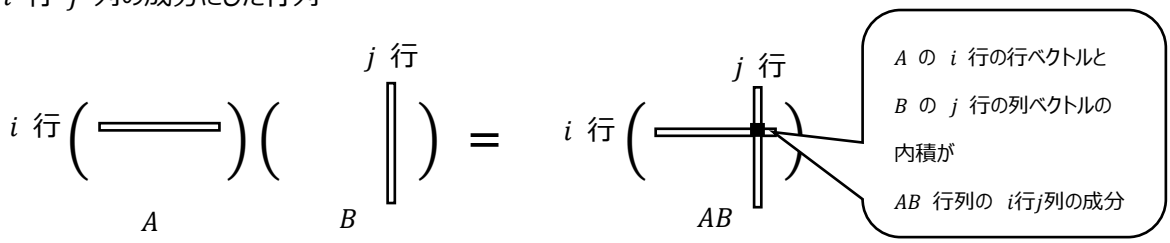
例＞

$A = \begin{pmatrix} 2 & 7 \\ 1 & 3 \end{pmatrix}$ 、 $B = \begin{pmatrix} 3 & 2 \\ 1 & 2 \end{pmatrix}$ のとき

$$A + B = \begin{pmatrix} 2+3 & 7+2 \\ 1+1 & 3+2 \end{pmatrix} = \begin{pmatrix} 5 & 9 \\ 2 & 5 \end{pmatrix} \qquad A - B = \begin{pmatrix} 2-3 & 7-2 \\ 1-1 & 3-2 \end{pmatrix} = \begin{pmatrix} -1 & 5 \\ 0 & 1 \end{pmatrix}$$

$$3A = \begin{pmatrix} 3 \times 2 & 3 \times 7 \\ 3 \times 1 & 3 \times 3 \end{pmatrix} = \begin{pmatrix} 6 & 21 \\ 3 & 9 \end{pmatrix}$$

・積　　行列 A, B の 積は A の i 行を行ベクトルとみなし、 B の j 列を列ベクトルとみなしたとき、それらの内積を i 行 j 列の成分にした行列



例＞

$A = \begin{pmatrix} 2 & 7 \\ 1 & 3 \end{pmatrix}$ 、 $B = \begin{pmatrix} 3 & 2 \\ 1 & 2 \end{pmatrix}$ のとき

$$AB = \begin{pmatrix} 2 & 7 \\ 1 & 3 \end{pmatrix}\begin{pmatrix} 3 & 2 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 2 \times 3 + 7 \times 1 & 2 \times 2 + 7 \times 2 \\ 1 \times 3 + 3 \times 1 & 1 \times 2 + 3 \times 2 \end{pmatrix} = \begin{pmatrix} 13 & 18 \\ 6 & 8 \end{pmatrix}$$

$$BA = \begin{pmatrix} 3 & 2 \\ 1 & 2 \end{pmatrix}\begin{pmatrix} 2 & 7 \\ 1 & 3 \end{pmatrix} = \begin{pmatrix} 3 \times 2 + 2 \times 1 & 3 \times 7 + 2 \times 3 \\ 1 \times 2 + 2 \times 1 & 1 \times 7 + 2 \times 3 \end{pmatrix} = \begin{pmatrix} 8 & 27 \\ 4 & 13 \end{pmatrix}$$

この例からわかるように、交換法則は成り立ちません。

$$AB \neq BA$$

単位行列 E においては、

$$AE = EA = A$$

が、成立します。

単位行列 E は、１と同じ性質を持つ行列です。

・アダマール積　　同じ形の行列（行と列が同じ数の行列）A, B において、同じ位置の成分を掛け合わせてできた行列を、行列A, B のアダマール積といい、記号 $A \odot B$ で表現します。

例＞

$A = \begin{pmatrix} 2 & 7 \\ 1 & 3 \end{pmatrix}$ 、 $B = \begin{pmatrix} 3 & 2 \\ 1 & 2 \end{pmatrix}$ のとき

$$A \odot B = \begin{pmatrix} 2 \times 3 & 7 \times 2 \\ 1 \times 1 & 3 \times 2 \end{pmatrix} = \begin{pmatrix} 6 & 14 \\ 1 & 6 \end{pmatrix}$$

・転置行列　　行列 A の i 行 j 列にある値を、j 行 i 列 に置き換えて得られた行列を、行列 A の 転置行列といい、記号 A^t や ${}^tA$ などと表記されます。

$A = \begin{pmatrix} 2 & 7 \\ 1 & 3 \end{pmatrix}$ のとき　　${}^tA = \begin{pmatrix} 2 & 1 \\ 7 & 3 \end{pmatrix}$

$B = \begin{pmatrix} 2 & 1 & 5 \\ 4 & 2 & 3 \end{pmatrix}$ のとき　　${}^tB = \begin{pmatrix} 2 & 4 \\ 1 & 2 \\ 5 & 3 \end{pmatrix}$

※最初の連立漸化式ですが、行列を使って次のように表現できます。

$$\begin{cases} a_{n+1} = a_n + 2b_n + 2 \\ b_{n+1} = 2a_n + 3b_n + 1 \end{cases}$$

$$\begin{pmatrix} a_{n+1} \\ b_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 2 & 2 \\ 2 & 3 & 1 \end{pmatrix}\begin{pmatrix} a_n \\ b_n \\ 1 \end{pmatrix}$$

６．微分・偏微分

AI が自ら学習するということの数学的意味は、学習データと一致するように設定値を「最適化」するということです。
その最適化に微分法は不可欠です。微分ってどのようなものか、概念を知っていきましょう。
まずは、微分の定義です。
関数$y = f(x)$ に対して、導関数 $f'(x)$ は次のように定義されます。この導関数を求めることを微分するといいます。

$$f'(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

※ $\lim_{\Delta x \to 0}$(式) は数 Δx を限りなく 0 に近づけたとき、(式) の近づく値を意味します。
※ 文字式や数式、関数に ′ （プライム記号）を付けて導関数を表します。

たとえば、

$f(x) = 3x$ のとき

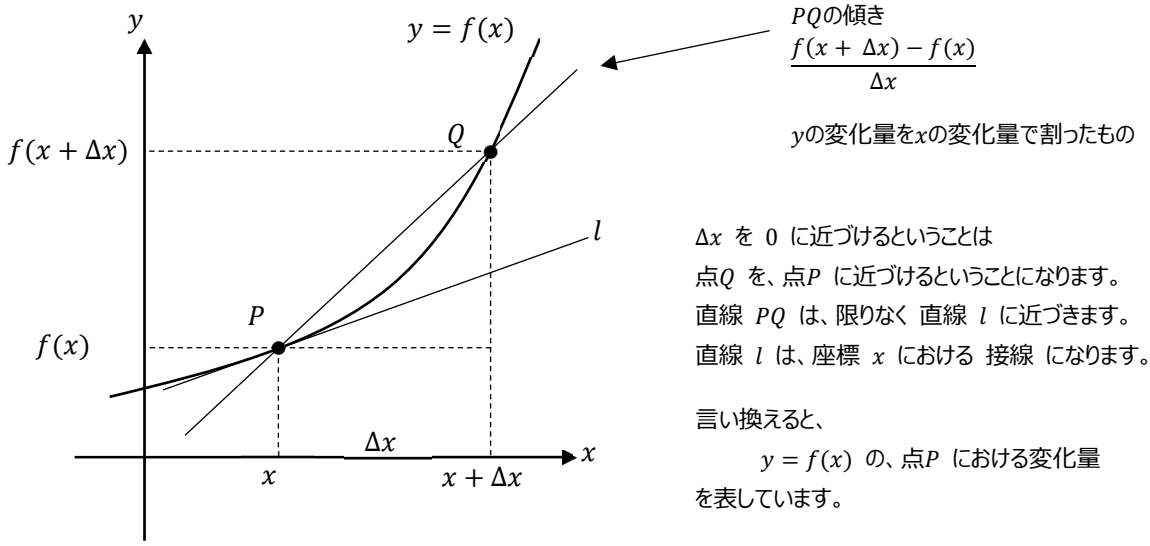
$$f'(x) = \lim_{\Delta x \to 0} \frac{3(x + \Delta x) - 3x}{\Delta x} = \lim_{\Delta x \to 0} \frac{3x + 3\Delta x - 3x}{\Delta x} = \lim_{\Delta x \to 0} \frac{3\Delta x}{\Delta x} = \lim_{\Delta x \to 0} 3 = 3$$

$f(x) = x^2$ のとき

$$f'(x) = \lim_{\Delta x \to 0} \frac{(x + \Delta x)^2 - x^2}{\Delta x} = \lim_{\Delta x \to 0} \frac{x^2 + 2x\Delta x + \Delta x^2 - x^2}{\Delta x} = \lim_{\Delta x \to 0} \frac{2x\Delta x + \Delta x^2}{\Delta x}$$

$$= \lim_{\Delta x \to 0} 2x + \Delta x = 2x$$

これが何を意味するのか？　以下のグラフを見てください。



微分の概念とは、通常変化量を求めるには最低 2 点の情報が必要なのですが、1 点の変化量を、極限の世界で求めようというものです。
AI でも、この極限の計算をするのか？　というと、ほとんどしません。通常は、微分の公式　を使っていきます。
ただ、概念を知らないと、式の意味が分からなくなることがあります。概念はおさえておきましょう。

・主な公式　（x を変数、 c を定数とします。）

$(c)' = 0$　　$(x)' = 1$　　$(x^n)' = nx^{n-1}$

$(e^x)' = e^x$　　$(e^{-x})' = -e^{-x}$

※ e は、ネイピア数（$y = e^x$ において、$x = 0$ での接線の傾きが　1　になる。）

・微分記号

関数 $y = f(x)$ の導関数を $f'(x)$ と表現しましたが、次のように分数で表現できます。

$$f'(x) = \frac{dy}{dx}$$

y の式を 変数である x について、微分するという意味になります。

・微分の性質

$$\{f(x) \pm g(x)\}' = f'(x) \pm g'(x)$$

$$\{cf(x)\}' = cf'(x) \qquad (c \text{ は定数})$$

例＞

$f(x) = 2x^2 + 3x + 1$ のとき

$$f'(x) = (2x^2)' + (3x)' + (1)' = 2(x^2)' + 3(x)' + (1)' = 2(2x) + 3(1) + 0 = 4x + 3$$

・分数関数

関数が分数形をしているときの微分に公式があります。

$$\left\{\frac{1}{f(x)}\right\}' = -\frac{f'(x)}{\{f(x)\}^2}$$

例＞

$$f(x) = \frac{1}{2x+3}$$

このとき、 $g(x) = 2x + 3$ とすると、

$$f(x) = \frac{1}{g(x)}$$

となります。微分をすると

$$f'(x) = \left\{\frac{1}{g(x)}\right\}' = -\frac{g'(x)}{\{g(x)\}^2} = \frac{-2}{(2x+3)^2} = \frac{-2}{4x^2 + 12x + 9}$$

と求めることができます。

AI でよく使う シグモイド関数 $\sigma(x)$ は次のように定義されています。

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$f(x) = 1 + e^{-x}$ とすると、

$$\sigma'(x) = \left\{\frac{1}{f(x)}\right\}' = -\frac{f'(x)}{\{f(x)\}^2} = -\frac{(1 + e^{-x})'}{(1 + e^{-x})^2} = \frac{e^{-x}}{(1 + e^{-x})^2}$$

※公式 $(e^{-x})' = -e^{-x}$ を使っています。

この式を少し変形します。

分子に 1 を加えて 1 を引きます。結果は 0 になり、式に影響はありません。

$$\frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1 + e^{-x} - 1}{(1 + e^{-x})^2} = \frac{1 + e^{-x}}{(1 + e^{-x})^2} - \frac{1}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} - \frac{1}{(1 + e^{-x})^2}$$

$$= \sigma(x) - \sigma(x)^2 = \sigma(x)(1 - \sigma(x))$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

※微分が、元の式のみで計算ができます。

関数 $y = f(x)$ において、 x を独立変数、 y を従属変数といいます。

今までは、関数として独立変数が、１つの場合を考えました。

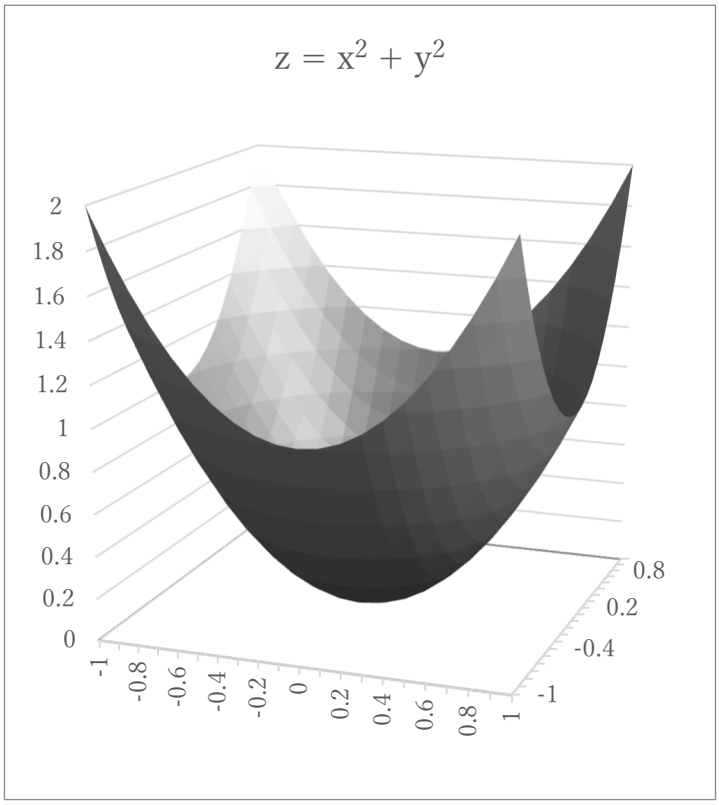
ここでは、独立変数が２つ以上の場合を考えていきます。このような関数を、 多変数関数 と、いいます。

たとえば、

$$z = x^2 + y^2$$

グラフにすると右図のようになります。

※$-1 \leqq x \leqq 1, -1 \leqq y \leqq 1$ の範囲のグラフになりますので、グラフはその範囲内のデータで構成されています。



AI で使用する関数を記述する変数の個数は何万にも及ぶ場合があります。

視覚的に関数を理解するのは不可能になりますが、１つの変数の場合を理解して、一般的な式を理解することはできます。

1 変数関数を表す記号は

$f(x)$ でした。

2 変数 x, y を独立変数とする関数は、

$f(x, y)$ と表します。

一般的に

n 変数 $x_1, x_1, \cdots, x_n$ を独立変数とする関数は

$f(x_1, x_1, \cdots, x_n)$ と表します。

多変数関数の場合も、微分法が適用できます。

ただし、どの変数に対して微分をするのかを明示する必要があります。ある特定の変数に対して微分することを 偏微分といいます。

たとえば、2 変数 x, y から成り立つ関数 $z = f(x, y)$ を考えてみましょう。

変数 x だけに着目して y は定数として考える微分を「x についての偏微分」と呼び、以下のように表現します。

$$\frac{\partial z}{\partial x} = \frac{\partial f(x, y)}{\partial x} = \lim_{\Delta x \to 0} \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x}$$

y についての偏微分も同様です。

$$\frac{\partial z}{\partial y} = \frac{\partial f(x, y)}{\partial y} = \lim_{\Delta y \to 0} \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y}$$

例＞

$z = wx + b$ のとき

$$\frac{\partial z}{\partial x} = w, \quad \frac{\partial z}{\partial w} = x, \quad \frac{\partial z}{\partial b} = 1$$

$f(x, y) = 3x^2 + 4y^3$ のとき

$$\frac{\partial f(x,y)}{\partial x} = 6x, \qquad \frac{\partial f(x,y)}{\partial y} = 12y^2$$

関数$y = f(u)$ があり、その u が $u = g(x)$ と表されるとき、 y は x の関数として、
$y = f(g(x))$ と表現することができます。
このような入れ子構造の関数 $f(g(x))$ を関数 $f(u)$ と $g(x)$ の 合成関数 といいます。
たとえば、
複数の入力 $x_1, x_2, \cdots, x_n$ に対して、$a(x)$ を関数として

$$y = a(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$$

は、次のような合成関数と考えられます。

$$\begin{cases} z = f(x_1,\ x_2, \cdots, x_n) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b \\ y = a(z) \end{cases}$$

・合成関数の微分公式
1 変数の関数 $y = f(u)$ があり、その u が 1 変数の関数 $u = g(x)$ と表されるとき、合成関数 $f(g(x))$ の導関数は次のようになります。

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

これを、1 変数関数の合成関数の 微分公式 と呼びます。また、チェーンルール や 連鎖律 などとも呼ばれます。
合成関数が 2 つ以上の場合も同様です。
例＞
y が u の関数、u が v の関数、v が x の関数のとき

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dv}\frac{dv}{dx}$$

のように表現します。
多変数関数のときにも、合成関数の微分公式がそのまま適用できます。ただし、関係するすべての変数について微分公式を適用する必要があります。
例＞
変数 z が u, v の関数で、 u, v がそれぞれ x, y の関数なら、 z は x, y の関数です。
$z = f(u, v), \quad u = g(x, y), \quad v = h(x, y)$ ならば、 $z = f(g(x, y), h(x, y))$ となります。
このとき次の公式（多変数のチェーンルール）が成り立ちます。

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u}\frac{\partial u}{\partial x} + \frac{\partial z}{\partial v}\frac{\partial v}{\partial x}$$

同様に

$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial u}\frac{\partial u}{\partial y} + \frac{\partial z}{\partial v}\frac{\partial v}{\partial y}$$

となります。